

Correction détaillée — Fonctions en Python (1)**Exercice 1***Fonction `exercice1` :*

```
1 def exercice1(a):  
2     return a**2 - a + 1
```

- 1. La fonction possède un **seul argument**, noté **a**.
- 2. La fonction renvoie la valeur du calcul $a^2 - a + 1$.
 - a. `exercice1(2)` = $2^2 - 2 + 1 = 4 - 2 + 1 = \mathbf{3}$.
 - b. `exercice1(5)` = $5^2 - 5 + 1 = 25 - 5 + 1 = \mathbf{21}$.
 - c. `exercice1(2.1)` = $2,1^2 - 2,1 + 1 = 4,41 - 2,1 + 1 = \mathbf{3,31}$.

La console affiche donc :

```
>>> exercice1(2)  
3  
>>> exercice1(5)  
21  
>>> exercice1(2.1)  
3.31
```

- **Remarque.** Avec un argument entier, le résultat est un entier ; avec un argument décimal (2.1), le résultat est décimal.

Exercice 2*Fonction `vitesse` :*

```
1 def vitesse(distance, temps):  
2     return distance / temps
```

- 1. La fonction possède **deux arguments** : **distance** et **temps**.
- 2. L'instruction `vitesse(120, 1.5)` calcule $120 \div 1,5 = 80$. La distance étant en kilomètres et le temps en heures, le quotient $\frac{\text{km}}{\text{h}}$ est une **vitesse en km/h**. Le résultat **80** signifie donc qu'un véhicule parcourant 120 km en 1,5 h roule à une vitesse moyenne de **80 km/h**.
- 3. On utilise la relation $v = \frac{d}{t}$, soit $t = \frac{d}{v}$. La fonction **temps** prend en arguments une distance **d** et une vitesse **v** et renvoie leur quotient :

```
1 def temps(d, v):  
2     return d / v
```

Par exemple, `temps(120, 80)` renvoie 1,5 (heure).

Exercice 3

Compléter la fonction *tension* (résistance R en ohm, intensité I en ampère).

- **Loi d'Ohm.** La tension aux bornes d'un appareil est le produit de sa résistance par l'intensité qui le traverse : $U = R \times I$ (en volts). La fonction prend donc deux arguments, R et I , et renvoie leur produit :

```
1 def tension(R, I):
2     return R * I
```

Par exemple, `tension(220, 0.5)` renvoie 110 (volts).

Exercice 4

Fonction *sphere* :

```
1 from math import pi
2 def sphere(R):
3     return 4/3 * pi * R**3
```

- 1. L'instruction de la première ligne, `from math import pi`, importe la constante π depuis le module `math`. Sans elle, le nom `pi` serait inconnu de Python et le calcul du volume provoquerait une erreur.

- 2. Le volume d'une sphère est $V = \frac{4}{3}\pi R^3$. Arrondir à 10^{-3} près, c'est garder 3 chiffres après la virgule : on encadre le calcul par `round(..., 3)`.

```
1 from math import pi
2 def sphere(R):
3     return round(4/3 * pi * R**3, 3)
```

Par exemple, `sphere(1)` renvoie alors 4,189 (au lieu de 4,18879...).

- **Remarque.** En Python, `4/3` est une division décimale qui vaut `1,333...` : la formule est donc correcte.

Exercice 5

Fonction *exercice5* :

```
1 from random import randint
2 def exercice5():
3     return randint(1, 6)
```

- 1. La fonction ne possède **aucun argument** : les parenthèses de `exercice5()` sont vides.
- 2. `randint(1, 6)` renvoie un entier **tiré au hasard** entre 1 et 6 (bornes comprises). En l'utilisant dix fois, on obtient dix entiers, par exemple :

```
3 1 6 6 2 4 5 1 3 2
```

Les résultats **changent à chaque exécution** : c'est le propre du hasard.

- 3. Cette fonction simule le **lancer d'un dé à 6 faces** parfaitement équilibré : chaque appel correspond à un lancer.

- 4. Une pièce parfaitement équilibrée n'a que **deux issues** (pile ou face). Il suffit de tirer au hasard entre deux valeurs, par exemple avec `randint(1, 2)` :

```
1 from random import randint
2 def piece():
3     if randint(1, 2) == 1:
4         return "Pile"
5     else:
6         return "Face"
```

- **Rôle de la première ligne.** `from random import randint` importe la fonction `randint` du module `random` ; sans elle, on ne pourrait pas tirer de nombre au hasard.

Exercice 6

Cube $ABCDEFGH$ de côté c ; L est le centre de la face $EFGH$ et $ABCDL$ la pyramide inscrite.

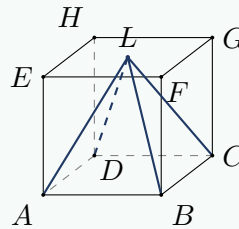
- **1. Volume du cube.** Le volume d'un cube de côté c est c^3 :

```
1 def cube(c):
2     return c**3
```

- **2. Volume de la pyramide $ABCDL$.** Le volume d'une pyramide est

$$V = \frac{1}{3} \times (\text{aire de la base}) \times (\text{hauteur}).$$

Ici, la base $ABCD$ est une face du cube, d'aire c^2 . Le sommet L est le centre de la face *opposée* $EFGH$: sa distance au plan de la base est donc la hauteur du cube, soit c .



On obtient donc $V = \frac{1}{3} \times c^2 \times c = \frac{c^3}{3}$:

```
1 def pyramide(c):
2     return c**3 / 3
```

- **En réutilisant la fonction cube.** Le volume de la pyramide vaut le tiers de celui du cube. On peut donc faire *appel* à la fonction déjà construite :

```
1 def pyramide(c):
2     return cube(c) / 3
```

Exercice 7

Écrire une fonction *solde* qui renvoie le prix soldé d'un article à partir du prix initial et de la remise en pourcentage.

- **Idée.** Appliquer une remise de $r\%$ revient à **multiplier le prix par** $\left(1 - \frac{r}{100}\right)$. La fonction prend deux arguments : le prix initial et la remise.

```
1 def solde(prix, remise):
2     return prix * (1 - remise / 100)
```

- **Exemple.** `solde(80, 25)` calcule $80 \times \left(1 - \frac{25}{100}\right) = 80 \times 0,75 = 60$: un article à 80 € soldé à 25 % coûte **60 €**.

- **Variante avec saisie au clavier.** Si l'on préfère que l'utilisateur saisisse les valeurs dans la console :

```
1 def solde():
2     prix = float(input("Prix initial : "))
3     remise = float(input("Remise en % : "))
4     return prix * (1 - remise / 100)
```